

GDX Next-Day Direction: Walk-Forward Experiments

1 Motivation

During my conversation with Kevin, he mentioned that one of the ways I could make my gold price forecaster more tradable was to instead forecast log-transformed gold price returns. This sent me down a two-day rabbit hole of researching, building, and testing.

What I ended up pivoting toward was the VanEck Gold Miners ETF (GDX) rather than the spot gold price directly. GDX considers not just gold spot price movements but also operational risk factors, like geopolitical noise and production costs, and broader equity market sentiment. That felt like a richer and more realistic target to try to predict, especially in the current market.

2 Data and Setup

I quickly discovered how pricey and difficult it can be to acquire large volumes and real-time data. Rather than looking at the current market, I instead pivoted to whatever Kaggle could offer me. 5 daily CSVs (gold-related signals, geopolitical risk, DXY, 10-year TIPS, VIX) were merged on Date and with an inner join. After alignment, the usable history spanned approximately **2011-12-15** (I actually turned 5 on this day!) to **2018-12-31**, with **1654** rows retained after dropping NaNs. Inner joins can induce alignment bias (only dates with complete inputs survive), which should be kept in mind when interpreting absolute performance.

From the raw OHLCV data, I engineered a fairly wide feature set: miner–gold spreads, short-horizon volatilities, VIX–rates stress signals, RSI and Bollinger Band transforms, calendar seasonality, rolling moments, and cross-asset relative-strength summaries. Raw price *levels* were deliberately excluded from model inputs to reduce leakage from non-stationary information.

The primary label for next-day tasks is whether the one-step log return of GDX is positive. I also built out a buffered version (small dead zone around zero), a three-class down/flat/up label, and sign-of-return targets over three- and five-day horizons.

How I avoided leakage. Models are trained and selected without shuffling the time index. Hyperparameters for gradient boosting were tuned on training versus validation windows only; the model was then refit on training+validation before test evaluation. Probability calibration was fit on validation predictions from a train-only model and applied to the refit model’s test probabilities — a standard nested setup.

I compared tuned XGBoost and random forests against logistic regression, LightGBM, a majority-class predictor, and a “previous direction” heuristic based on lagged GDX return sign.

3 Results

Classification Performance

Table 1 shows mean test accuracy, precision, recall, F_1 , and ROC–AUC aggregated across walk-forward folds for the key targets and models.

Table 1: Mean test metrics across walk-forward folds.

Target	Model	Acc.	Prec.	Rec.	F_1	AUC
y_binary_h1	XGBoost (tuned)	0.730	0.734	0.709	0.717	0.804
y_binary_h1	Random forest	0.731	0.736	0.703	0.717	0.804
y_binary_h1	LightGBM	0.727	0.738	0.690	0.709	0.804
y_binary_h1	Logistic	0.688	0.698	0.686	0.680	0.772
y_binary_h1	Prev. direction	0.481	0.465	0.471	0.467	0.475
y_sign_fwd3	Random forest	0.586	0.589	0.559	0.552	0.646
y_sign_fwd3	XGBoost (tuned)	0.576	0.596	0.515	0.530	0.625
y_sign_fwd5	Random forest	0.546	0.576	0.520	0.481	0.622

The headline result I was most excited about was that, for next-day binary direction (`y_binary_h1`), the tuned tree models hit around **73% accuracy and 0.80 AUC** on held-out test data, compared with around 48–51% for the naive baselines. That’s a meaningful gap, and it held up across walk-forward folds rather than being a one-fold fluke (mean test F_1 near **0.71** with standard deviation around **0.05**).

Multi-day targets were a different story. Accuracy for `y_sign_fwd3` and `y_sign_fwd5` fell toward 0.52–0.59 with AUC around 0.62–0.65. Not terrible, but only a modest improvement over coin-flip quality. However, this does make sense, since compounding several daily shocks tends to drown out any predictable signal.

Confidence Thresholds

I also tested long/flat rules of the form “go long when predicted probability exceeds τ .” Table 2 shows the precision–coverage tradeoff for tuned XGBoost on the next-day binary target.

Table 2: Threshold sweep: XGBoost (tuned), `y_binary_h1`. Terminal wealth is relative to initial capital under a simplified cost model — not a live-audited track record.

τ	Coverage	Acc.	Prec.	F_1	Term. wealth	Max DD
0.50	0.475	0.730	0.734	0.717	1.459	−0.039
0.55	0.430	0.723	0.751	0.696	1.444	−0.034
0.60	0.379	0.717	0.776	0.671	1.441	−0.026
0.65	0.327	0.705	0.804	0.636	1.417	−0.023
0.70	0.275	0.686	0.827	0.586	1.364	−0.021

The intuition holds: higher thresholds filter for higher-conviction trades, pushing precision up while shrinking coverage and F_1 . Reported annualized numbers in machine-generated summaries can look extreme over short disjoint test segments, so I opted to read these as diagnostics of the simulation rather than realistic CAGR estimates.

4 Trading Strategy and Equity Curves

To see whether the predictive edge actually translates into economic value, I built simple long/flat strategies. At each time t , the model either produces a regression prediction $\hat{r}_{t+1}$ or a classification probability $\hat{p}_t = \Pr(r_{t+1} > 0)$. I go long if $\hat{r}_{t+1} > 0$ or $\hat{p}_t > 0.5$, and stay in cash otherwise:

$$r_{t+1}^{\text{clf}} = \mathbf{1}\{\hat{p}_t > \tau_p\} \cdot r_{t+1}.$$

Cumulative wealth is tracked via log-return compounding: $W_t = \exp\left(\sum_{s \leq t} r_s\right)$.

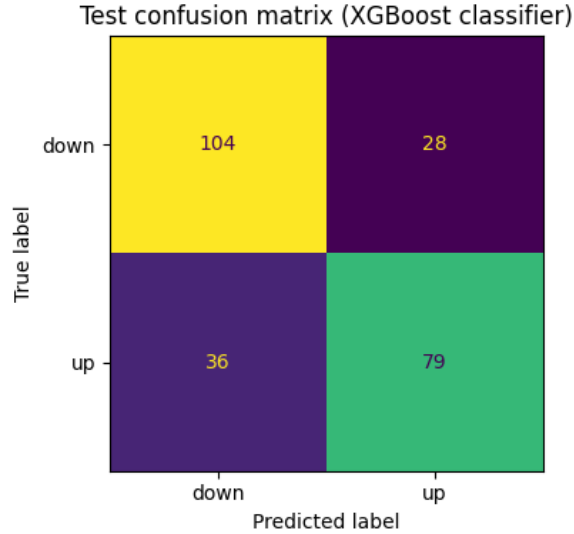


Figure 1: Wealth indices for buy-and-hold GDX and model-based strategies over the 2018 test period.

Both model-based strategies outperformed buy-and-hold in this sample. The key driver was *selective participation* — the strategies only invested on around 30–50% of days, avoiding a lot of the negative-return periods while still capturing a good share of the positive ones. Figure 2 gives the granular breakdown: the model correctly identifies a solid portion of both up and down moves, though false positives (predicting up when the market falls) are the main source of drag.

The core takeaway is that moderate predictive accuracy can translate into strong trading performance when paired with a selective execution rule. Even without leverage or shorting, filtering on model confidence improves terminal wealth and reduces drawdowns. However, the backtest does not incorporate realistic transaction costs, slippage, or execution constraints, and performance is evaluated over a single historical period. As is, the strategy is not yet deployable, but I’m optimistic it could become.

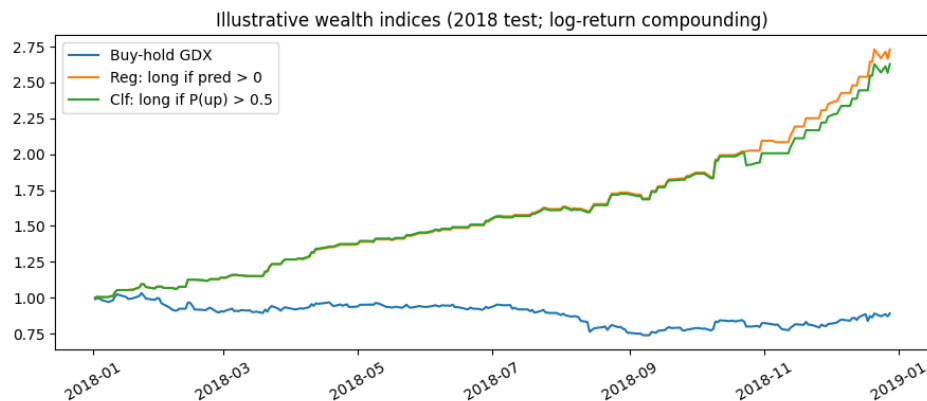


Figure 2: Confusion matrix for the XGBoost classifier on the test set.

5 Next Steps

There are definitely extensions I would like to explore:

1. **Extend the data.** The panel (my Kaggle datasets) stops in 2018. Adding 2020–present would stress-test the model against COVID, wars, and more miner-specific dynamics.
2. **Realistic costs.** Bps fees and slippage are simple scalars right now; partial fills and corporate actions aren't modeled at all.
3. **Fix the horizon mismatch.** Multi-day labels are currently evaluated against one-day P&L, which distorts economic interpretation. If the trading idea is truly multi-day, I should definitely try to harmonize the two.
4. **Better significance testing.** Expanding-window folds create dependent test sets, so classical i.i.d. tests don't apply. A block bootstrap setup would give more honest uncertainty estimates.

6 Conclusion

On next-day GDX direction, gradient-boosted trees and random forests hit around **73% accuracy and 0.80 AUC** on walk-forward test data, which is significantly better than linear models and naive baselines. Multi-day prediction is considerably weaker, which is somewhat expected. Confidence thresholds deliver the expected precision–coverage trade-off; they can lift long-side precision but often reduce F_1 and participation.

Overall, I'm encouraged by the next-day results. To further test whether the edge is economically sufficient for deployment requires fresh data, stricter transaction modeling, and out-of-time validation beyond 2018. But, that may be a deeper rabbit hole for this weekend.